



PDF Parsing Is Harder Than You Think

Your extractor returned text. That does not mean it returned the document.



PDF Describes Ink, Not Meaning

HTML says "this is a heading". PDF says "put a glyph here".

HTML

```
<h1>Q3 Summary</h1>
<table>
  <tr><td>Revenue</td>
    <td>4.2</td></tr>
</table>
```

Structure is explicit

PDF

```
BT /F1 18 Tf 72 720 Td
  (Q3 Summary) Tj ET
BT /F2 11 Tf 72 690 Td
  (Revenue) Tj ET
BT /F2 11 Tf 180 690 Td (4.2) Tj
ET
```

Structure must be inferred from x/y and font size

There is often no paragraph, no table, no reading order and no heading stored anywhere in the file. Those are things a human infers by looking at the page, and things your parser has to **reconstruct from coordinates**.



Every PDF parser is guessing. Better parsers guess better. None are reading structure that was reliably recorded, because for most PDFs it never was.



Three Kinds Of PDF, Three Pipelines

Identify which you have before choosing a tool

TYPE	WHAT IS IN THE FILE	HOW TO HANDLE IT
DIGITAL NATIVE	A text layer exists, glyphs and coordinates stored in the file. Born from Word, LaTeX or web export.	Extract directly with a layout aware reader
SCANNED IMAGE	The page is a photo. Zero extractable text. Scanner output, photographed docs, old archives.	OCR is required, nothing else works
HYBRID	Some pages text, some scanned, or a text page with scanned tables. Signed contracts, forms with attachments.	Detect per page and route each one separately



Hybrids break pipelines. A document that mostly extracts fine, with three scanned pages in the middle, silently loses those pages. Nothing errors. The content is just not there.



Route Each Page, Not Each Document

One check per page is what keeps hybrids from leaking

```
import pymupdf # fitz

def classify_page(page) → str:
    text = page.get_text().strip()
    # Area covered by images vs total page area
    boxes = [page.get_image_bbox(i) for i in page.get_images(True)]
    img_area = sum(b.width * b.height for b in boxes)
    page_area = page.rect.width * page.rect.height
    cover = img_area / page_area if page_area else 0

    if len(text) < 50 and cover > 0.5: return "scanned"
    if len(text) < 50: return "empty"
    if cover > 0.3: return "hybrid"
    return "digital"
```

PAGE 1 TO 2

digital, straight to text extraction

OK

PAGE 3

scanned, must go through OCR

WOULD BE LOST

PAGE 4

hybrid, extract text plus OCR the image regions

SPLIT



A document level check calls this file "digital" and drops page 3 entirely, with no error anywhere.



AI Coach John

Follow



 Repost

Drawing Order Is Not Reading Order

Glyphs come out in whatever order the generator emitted them

📄 PAGE LAYOUT

+-----+	
	HEADER
+-----+	
	Column A Column B
	text ... text ...
+-----+	
	SIDEBAR
+-----+	
	FOOTNOTE FOOTER
+-----+	

☰ NAIVE EXTRACTION ORDER

1. header text
2. left column line 1
3. right column line 1 <
4. left column line 2
5. right column line 2 <
6. sidebar
7. footnote
8. footer

Columns interleaved line by line into meaningless prose



Sorting does not fix it. Top to bottom then left to right works on two clean columns and fails on spanning headers, floating figures and unequal column heights. Reading order is layout analysis, not sorting.



Work With Blocks And Coordinates

Detect the gutters, then read each column top to bottom

```
def extract_in_reading_order(page, gap=50):
    blocks = page.get_text("blocks") # x0,y0,x1,y1,text,no,type
    tb = [b for b in blocks if b[6] == 0 and b[4].strip()]
    if not tb: return ""

    # Vertical gutters: x ranges with no text across the page
    xs = sorted(b[0] for b in tb)
    gutters = [(a, b) for a, b in zip(xs, xs[1:]) if b - a > gap]

    if not gutters: # single column
        return "\n".join(b[4] for b in sorted(tb, key=lambda b: b[1]))

    # Split at the widest gutter, then read each side in turn
    split_x = max(gutters, key=lambda g: g[1] - g[0])[1]
    left = sorted([b for b in tb if b[0] < split_x], key=lambda b: b[1])
    right = sorted([b for b in tb if b[0] ≥ split_x], key=lambda b:
b[1])
    return "\n".join(b[4] for b in left + right)
```

Simple two column academic paper

GUTTERS WORK

Magazine layouts, forms with nested boxes, mixed direction scripts

HEURISTICS BREAK

Then reach for layout models: **unstructured**, **docling**, **marker**, **layoutparser**



AI Coach John

Follow



Repost

A Table Is Just Text At Coordinates

Most PDF tables carry no table structure at all

⚙ WHAT IS ACTUALLY IN THE FILE

"Region"	at (72, 600)	"Q1"	at (200, 600)
"EMEA"	at (72, 584)	"4.2"	at (200, 584)
"APAC"	at (72, 568)	"3.3"	at (200, 568)

A table exists only because the x coordinates line up. Your parser has to notice that.

MULTI-LINE

One logical row spans three text rows

MERGED

A header spanning two columns

BORDERLESS

No ruling lines to detect at all

NESTED

A table living inside a cell

SPLIT PAGES

Header repeats on page two, or it does not

FOOTNOTES

Superscript markers merge into cell values



Three Ways To Pull A Table Out

In increasing order of robustness

```
# 1. Line based detection
import pdfplumber
with pdfplumber.open(path) as pdf:
    tables = pdf.pages[0].extract_tables()
    # No ruling lines? Switch strategy to whitespace
    tables = pdf.pages[0].extract_tables(
        {"vertical_strategy": "text", "horizontal_strategy": "text"})

# 2. Dedicated table libraries
import camelot
t = camelot.read_pdf(path, pages="1-5", flavor="lattice") # ruled
t = camelot.read_pdf(path, pages="1-5", flavor="stream") # gaps
print(t[0].parsing_report) # includes an accuracy estimate

# 3. ML layout models, best for borderless and complex tables
# Detect the table region first, then rebuild rows and columns.
# Via unstructured, docling, or table-transformer directly.
```

Clean ruled tables, consistent format

PDFPLUMBER, CAMELOT LATTICE

Borderless but well aligned

CAMELOT STREAM

Varied layouts across many documents

ML LAYOUT MODELS



Validate whatever you choose. Table extraction fails silently and returns plausible looking wrong numbers, the worst error type for financial or medical documents.



AI Coach John

Follow



 Repost

Catching Silent Table Corruption

Three checks that find misalignment before it reaches your index

```
def validate_table(rows) → list[str]:
    if not rows: return ["empty_table"]
    problems, widths = [], [len(r) for r in rows]

    if len(set(widths)) > 1:
        problems.append(f"ragged_rows: {sorted(set(widths))}")

    for c in range(max(widths)):
        col = [(r[c] or "").strip() for r in rows if len(r) > c]
        # A mostly empty column means a split or merge error
        if sum(not v for v in col) / len(col) > 0.7:
            problems.append(f"column_{c}_mostly_empty")
        # Numeric columns should stay numeric
        num = sum(bool(re.fullmatch(r"[+]?[\\d.]+%?", v)) for v in
col[1:])
        if 0.5 < num / max(len(col) - 1, 1) < 0.95:
            problems.append(f"column_{c}_mixed_types")
    return problems
```

🌟 THE CHECK THAT SAVES YOU

Region	Q1	Q2
EMEA	4.2	5.1
APAC	3.3	3.9
LATAM	see	note 4
NA	6.1	6.8

< footnote leaked in
and shifted the rest

A mixed type column is almost always misalignment, not real data



When OCR Is The Only Option

No text layer means no shortcuts

```
import pytesseract
from pdf2image import convert_from_path

def ocr_pdf(path, dpi=300, lang="eng") → list[str]:
    # DPI matters more than almost any other setting
    pages = convert_from_path(path, dpi=dpi)
    return [pytesseract.image_to_string(p, lang=lang) for p in pages]
```

150 DPI

Small text turns ambiguous, digits suffer most: 8/6/0 and 1/1/1

300 DPI

The standard baseline for document OCR

400 TO 600

Worth it for small print, footnotes and dense tables

ABOVE 600

Diminishing returns, much slower, far more memory

LANG="ENG"

English only, and it will mangle accented text

LANG="ENG+DEU"

Multiple languages, listed in priority order



Rendering low to "save time" is the top cause of poor OCR, and the wrong language pack quietly destroys every accent. **Compute is cheaper than reprocessing a corpus.**



AI Coach John

Follow



 Repost

Image Quality Beats Engine Choice

Four cheap steps that lift OCR accuracy more than a new model

```
def preprocess_for_ocr(image):  
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # 1  
  
    # 2. Deskew, scans are rarely perfectly straight  
    coords = np.column_stack(np.where(gray < 200))  
    angle = cv2.minAreaRect(coords)[-1]  
    angle = -(90 + angle) if angle < -45 else -angle  
    if abs(angle) > 0.5:  
        h, w = gray.shape  
        M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)  
        gray = cv2.warpAffine(gray, M, (w, h),  
borderMode=cv2.BORDER_REPLICATE)  
    # 3. Denoise, scanner speckle becomes stray punctuation  
    gray = cv2.fastNlMeansDenoising(gray, h=10)  
    # 4. Adaptive threshold, survives uneven lighting  
    return cv2.adaptiveThreshold(gray, 255,  
cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 31, 15)
```

GRayscale

Removes colour noise and halves the data

DESKEW

A two degree tilt degrades line detection and can merge adjacent lines

DENOISE

Speckle otherwise turns into stray punctuation

THRESHOLD

A shadow across the page makes a global threshold fail on the dark half



These matter most on photographed documents and old archives. **Test on your own scans** rather than applying all of it blindly.



AI Coach John

Follow



Repost

OCR Tells You When It Is Unsure

That signal is free, and almost nobody reads it

```
def ocr_with_confidence(image, min_conf=60) → dict:
    d = pytesseract.image_to_data(image,
                                   output_type=pytesseract.Output.DICT)
    words, low = [], []
    for text, conf in zip(d["text"], d["conf"]):
        conf = int(conf)
        if not text.strip() or conf < 0: continue
        words.append(text)
        if conf < min_conf: low.append((text, conf))
    confs = [int(c) for c in d["conf"] if int(c) ≥ 0]
    return {"text": " ".join(words),
            "mean_confidence": sum(confs) / len(confs),
            "low_confidence_ratio": len(low) / max(len(words), 1)}
```

ABOVE 85

Accept the page

SHIP**70 TO 85**

Accept, but flag it for a spot check

FLAG**BELOW 70**

Reject: re-run at higher DPI, change preprocessing, or send to review

REDO

Q ALSO WORTH CHECKING

- Low confidence ratio above 0.15 is a systematic problem, not noise
- Many single character "words" points to over thresholding



Tesseract is the common default. **PaddleOCR and cloud OCR often win on hard scans, non-Latin scripts and handwriting.** Benchmark on your own sample.



AI Coach John

[Follow](#)[Repost](#)

Text Extracted "Fine" Can Still Be Wrong

Six defects that survive a successful extraction

LIGATURES

"financial" is one glyph. It never matches a search for "financial".

NO TOUNICODE

Fonts embedded without a character map return glyph codes. Output looks like mojibake and often needs OCR.

SOFT HYPHENS

"require-" then "ments" across a line break becomes two unmatched tokens.

SUPERSCRIPTS

Footnote markers merge into words and numbers: "revenue³" or "4.21".

NO SPACES

Some generators position every word by coordinate: "Revenuegrew12percent".

RTL AND BIDI

Arabic and Hebrew may extract in visual order rather than logical order, reversed.

🔍 THE DIAGNOSTIC THAT CATCHES MOST OF THIS

Print 500 characters from 5 random documents and **read it**. Look for words run together, nonsense sequences, mid word hyphens, stray digits on numbers, or text not in the language you expect.



Headings Are Inferable From Fonts

Size relative to body text is the strongest signal you have

```
def infer_headings(page) → list[dict]:
    spans = []
    for block in page.get_text("dict")["blocks"]:
        for line in block.get("lines", []):
            for s in line["spans"]:
                if s["text"].strip():
                    spans.append({"text": s["text"].strip(),
                                "size": round(s["size"], 1), "bold":
                                bool(s["flags"] & 16)})
    # Body text is whatever size is most common on the page
    body = Counter(s["size"] for s in spans).most_common(1)[0][0]
    for s in spans:
        if s["size"] > body * 1.4: s["level"] = 1
        elif s["size"] > body * 1.15 or s["bold"]: s["level"] = 2
        else: s["level"] = None # body text
    return spans
```

1 PDF outline or bookmarks. Explicit and free: `doc.get_toc()`

2 Font size vs body text. Strong and widely applicable

3 Bold plus a short line. Good secondary signal

4 Numbering patterns. "3.2.1 Scope" is regex detectable

5 Vertical whitespace. Weakest, entirely layout dependent



Check for an outline first. Many PDFs from Word or LaTeX already store the full heading hierarchy.



Zero Text, And No Error

Forms, encryption, rotation and attachments

```
doc = pymupdf.open(path)

# FORM FIELDS live outside the text layer entirely
for page in doc:
    for w in page.widgets():
        print(w.field_name, "=", w.field_value)
# A filled form extracts as an EMPTY page without this

# ENCRYPTED or permission protected
if doc.needs_pass and not doc.authenticate(password):
    raise ValueError("cannot decrypt")
# Some open fine but forbid extraction, see doc.permissions

# ROTATED PAGES extract out of order, or not at all
for page in doc:
    if page.rotation: page.set_rotation(0)

# EMBEDDED ATTACHMENTS: the real data is often a spreadsheet
for i in range(doc.embfile_count()):
    print("attached:", doc.embfile_info(i)["filename"])
```

☰ SILENT ZERO CHECKLIST

Page came back empty? Check in this order:

- ✓ Is it scanned? Send it to OCR
- ✓ Are there form widgets? Read the field values
- ✓ Is the page rotated? Normalise the rotation
- ✓ Is extraction permitted? Check doc.permissions
- ✓ Are there embedded files? Extract them separately
- ✓ Genuinely blank? Fine, log it and move on



What To Reach For, **And When**

No single library covers a real corpus

TOOL	WHERE IT FITS
PYPDF	Pure Python, no system deps. Simple text PDFs, metadata, merge and split. Weak on layout.
PYMUPDF (FITZ)	Fast and layout aware, gives blocks and spans with coordinates and font info. The workhorse. Check its licence.
PDFPLUMBER	Best introspection: per character positions, ruling lines, tunable table strategies. Slower.
CAMELOT, TABULA	Dedicated table extraction with accuracy reporting. Tabula needs Java.
TESSERACT, PADDLEOCR	OCR engines. Benchmark both on your own documents, results vary by script and quality.
UNSTRUCTURED, DOCLING, MARKER	High level partitioning with layout models. Fastest path to structured output, less control.



The realistic answer is a routing pipeline, not one tool. Digital pages to a layout aware extractor, tables to a table path, scans to OCR, with per page classification deciding.



AI Coach John

Follow



PROTIBRIDGE
Solving For Better Future



Repost

Everything Assembled, Failures Logged

Never return silently empty content

```
def parse_pdf(path: str) → dict:
    doc = pymupdf.open(path)
    if doc.needs_pass and not doc.authenticate(""):
        return {"status": "encrypted", "pages": []}
    pages, warnings = [], []
    for i, page in enumerate(doc):
        if page.rotation: page.set_rotation(0)
        kind, conf = classify_page(page), None
        if kind == "scanned":
            r = ocr_with_confidence(render_page(page, dpi=300))
            text, conf = r["text"], r["mean_confidence"]
            if conf < 70: warnings.append(f"page {i}: low OCR conf")
        elif kind == "empty":
            text = extract_form_fields(page) if page.widgets() else ""
        else:
            text = extract_in_reading_order(page)
            text = normalise_text(fix_hyphenation(text))
        if kind != "empty" and len(text.strip()) < 20:
            warnings.append(f"page {i}: suspiciously little text")
        pages.append({"page": i, "kind": kind, "text": text, "conf":
            conf,
            "tables": extract_and_validate_tables(page)})
    return {"status": "ok", "pages": pages, "warnings": warnings}
```

🔔 WHAT A RUN SHOULD END WITH

```
47 pages: low OCR confidence
8 pages: suspiciously little text
3 files: encrypted, skipped
1 file: extraction not permitted
```



Surface Systematic Failure

Aggregate counts reveal what per document errors hide

```
def audit_corpus(parsed_docs) → dict:
    stats, suspicious = Counter(), []
    for doc in parsed_docs:
        for p in doc["pages"]:
            stats[p["kind"]] += 1
            text = p["text"]
            if not text.strip():
                stats["empty_output"] += 1; continue
            # Low alphanumeric ratio means garbage
            alnum = sum(c.isalnum() or c.isspace() for c in text) /
len(text)

            if alnum < 0.6: suspicious.append((p, "low_alnum"))
            # Very long "words" mean the spaces went missing
            if any(len(w) > 40 for w in text.split()):
                suspicious.append((p, "missing_spaces"))
            # Dictionary hit rate catches mojibake and bad OCR
            words = re.findall(r"[a-z]{3,}", text.lower())
            if words and sum(w in DICTIONARY for w in words) / len(words)
< 0.5:
                suspicious.append((p, "low_dictionary_hits"))

    return {"page_kinds": dict(stats), "suspicious": suspicious}
```

🔍 READ THE DISTRIBUTION, NOT JUST THE ERRORS

```
page_kinds: {digital: 8,204, scanned: 143,
             hybrid: 22, empty: 891}
```

891 empty pages is not "some blank pages". It is a **systematic failure**: one generator or one rotation case failing consistently. Investigate before you index.



Common Mistakes

What goes wrong in PDF pipelines

× EXTRACTION

- Never reading the extracted text
- Classifying per document, losing scanned pages
- Assuming `extract_text()` gives reading order
- Flattening tables into space separated strings
- Missing form values outside the text layer

× OCR AND OPS

- OCR at 150 dpi to save time
- Wrong language pack, accents destroyed
- Ignoring confidence scores entirely
- No outline check before inferring headings
- Swallowing exceptions, so nothing surfaces

SYMPTOM	LIKELY CAUSE
"text is gibberish"	missing ToUnicode map, or bad OCR
"words run together"	generator emitted no space characters
"sentences are scrambled"	multi column read in drawing order
"numbers are wrong"	table columns misaligned
"some files return nothing"	scanned pages, or permission flags
"fails only in prod"	missing system deps: poppler, tesseract



AI Coach John

Follow



Repost

Returning Text Is Not Extraction Working

The question is never "did it run", it is "did it recover the document"

DO THIS	AND YOU GET
CLASSIFY PER PAGE	hybrid documents stop losing pages
LAYOUT AWARE EXTRACT	columns read in the right order
TABLES ON THEIR OWN PATH	numbers stay attached to their labels
OCR AT 300 DPI OR MORE	scans become usable text
GATE ON CONFIDENCE	bad OCR gets caught, not indexed
NORMALISE CHARACTERS	"financial" finally matches "financial"
CHECK THE OUTLINE	free heading structure when it exists
WARN, NEVER SWALLOW	failures surface as a list to act on
READ THE OUTPUT	the check nothing else replaces



PDF stores ink, not meaning, so every parser is reconstructing what was never recorded. Follow AI Coach John for more production document engineering.

